

Inventory Management Platform — Complete UI Walkthrough

Application: Full-stack inventory management platform with immutable StockLedger architecture, role-based access control (Admin/Worker), and mobile-ready API.

Tech Stack: FastAPI · React · MongoDB · Tailwind · shadcn/ui

Live Preview: <https://stock-traceability-1.preview.emergentagent.com>

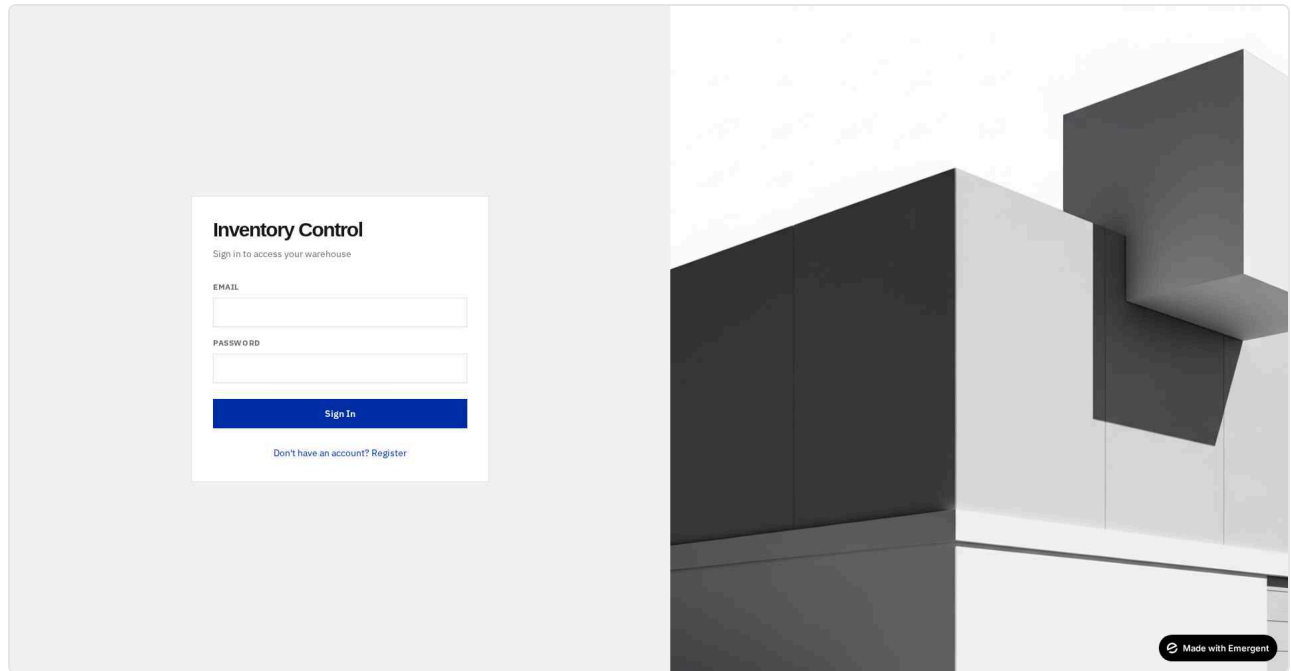
Admin Login: `admin@inventory.com` / `Admin@123`

Table of Contents

1. [Login Page](#)
 2. [Register Page](#)
 3. [Admin Dashboard](#)
 4. [Product Management](#)
 5. [Add Product Dialog](#)
 6. [Location Management](#)
 7. [Add Location Dialog](#)
 8. [Stock Ledger](#)
 9. [Record Transaction Dialog](#)
 10. [Worker Page](#)
 11. [Architecture Highlights](#)
-

1. Login Page

Route: `/login`



Purpose

Authenticated entry point for both Admin and Worker users.

Key Elements

- **Left panel:** Email + Password form with "Sign In" CTA and a link to Register.
- **Right panel:** Architectural warehouse hero image — minimalist Swiss design.
- **Branding:** "Inventory Control" displayed in Cabinet Grotesk (heavy, tight-tracking).

Behavior

- Backend returns the user's role on successful login.
- Role-based routing: `admin` → `/dashboard`, `worker` → `/worker`.
- JWT token is stored in `localStorage` + `httpOnly` cookie (dual-auth strategy for cross-origin scenarios).

Test Credentials

admin@inventory.com / Admin@123

2. Register Page

Route: `/register`

Create Account
Register for warehouse access

FULL NAME

EMAIL

PASSWORD

ROLE
Worker

Create Account

Already have an account? [Sign in](#)

Made with Emergent

Purpose

Self-service account creation.

Key Elements

- **Fields:** Full Name, Email, Password, **Role** dropdown.
- **Role options:** `Worker` (default) or `Admin`.
- Cross-link "Already have an account? Sign in" → routes to `/login`.

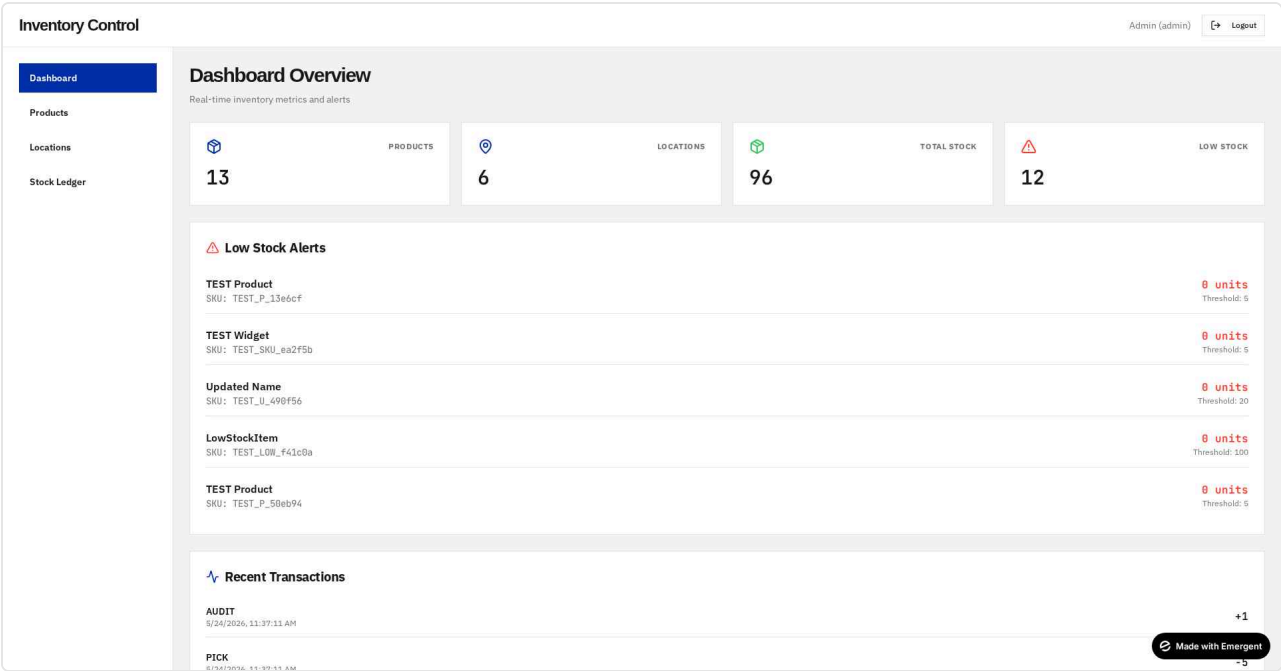
Behavior

- POST `/api/auth/register` → returns user object + access token in body.
- Auto-redirects based on role after successful registration.
- Email uniqueness is enforced server-side via MongoDB unique index.

 **Production note:** Admin role selection should be server-restricted in production (current self-serve admin signup is for demo convenience).

3. Admin Dashboard

Route: `/dashboard` (admin only)



Purpose

Real-time operational overview — stat cards, low-stock alerts, and a transaction feed.

Key Sections

A. Top Stat Cards (4 KPIs)

CARD	DESCRIPTION	SOURCE
Products	Total SKUs in catalog	<code>db.products.count_documents({})</code>
Locations	Total warehouse bins	<code>db.locations.count_documents({})</code>
Total Stock	Sum of all ledger entries	Aggregation: <code>SUM(stock_ledger.quantity_change)</code>
Low Stock	Products below threshold	Per-product <code><</code> comparison on aggregated stock

B. Low Stock Alerts

Lists every product whose current stock falls below its configured `low_stock_threshold` . Each row shows:

- Product name + SKU (JetBrains Mono)
- Current units (red, bold)
- Threshold value

C. Recent Transactions

Chronological feed of transaction events (`AUDIT` , `PICK` , `RECEIVE` , `TRANSFER`) with signed quantity deltas (`+1` , `-5` , `+100`) and timestamps.

D. Left Sidebar Navigation

- **Dashboard** (active — highlighted blue `#002FA7`)
 - Products
 - Locations
 - Stock Ledger
-

4. Product Management

Route: `/products` (admin only)

Inventory Control

Dashboard

Products

Locations

Stock Ledger

Product Management

Manage SKUs and inventory thresholds

+ Add Product

SKU	PRODUCT NAME	CURRENT STOCK	THRESHOLD	UNIT	ACTIONS
TEST_P_13e6cf	TEST Product	0	5	pcs	
TEST_SKU_ea2f5b	TEST Widget	0	5	pcs	
TEST_U_498f56	Updated Name	0	20	pcs	
TEST_LOW_f41c8a	LowStockItem	0	100	pcs	
TEST_P_50eb94	TEST Product	0	5	pcs	
TEST_SKU_d93a5c	TEST Widget	96	5	pcs	
TEST_U_a6c2d3	Updated Name	0	20	pcs	
TEST_LOW_aa0e6c	LowStockItem	0	100	pcs	
UI_SKU_5984	UI Test Product	0	15	pcs	
DEBUG_TEST_1779625834	Debug Product	0	5	units	
BEARER_TEST_1779625911	Bearer Test	0	5	units	
E2E_1779625936	E2E Test Product	0	10	units	

Purpose

CRUD interface for the SKU master data.

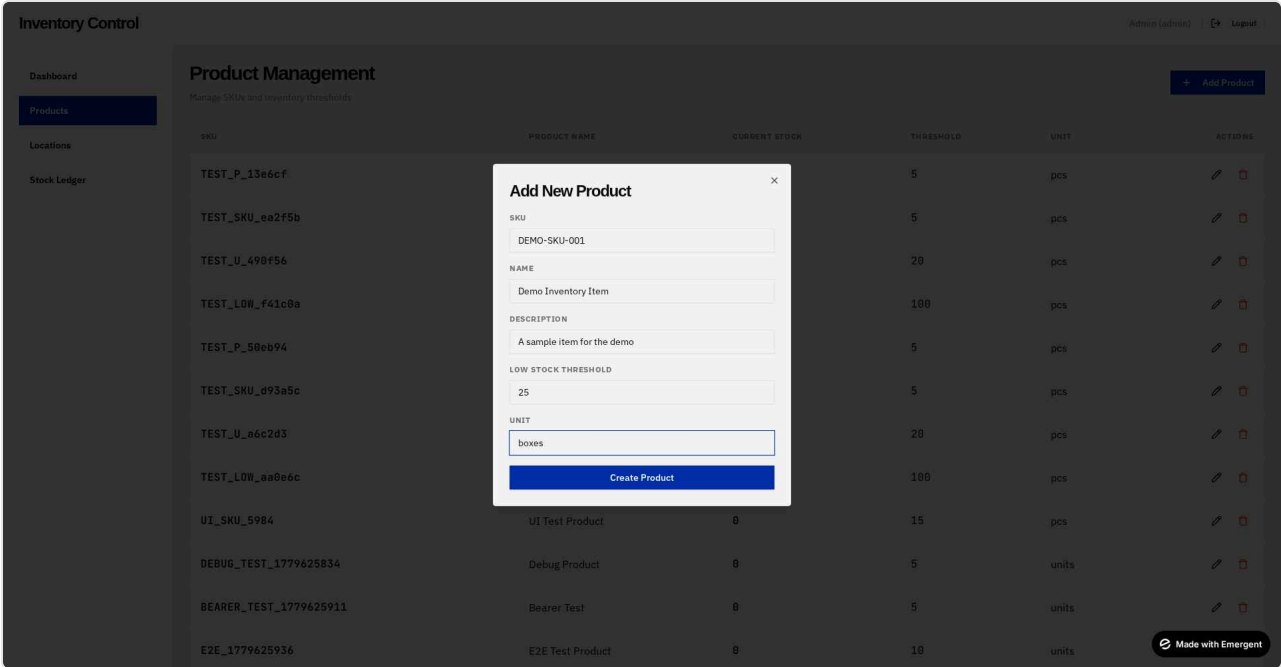
Table Columns

COLUMN	NOTES
SKU	Unique identifier — rendered in mono font
Product Name	Human-readable label
Current Stock	Computed live from <code>stock_ledger</code> (never stored on product!)
Threshold	Configurable low-stock alert level
Unit	e.g., <code>units</code> , <code>pcs</code> , <code>boxes</code>
Actions	Edit · Delete

Key Behaviors

- **Add Product** button (top-right, primary blue) opens the create dialog.
- Each row's `Current Stock` is the result of a MongoDB aggregation pipeline that sums `quantity_change` from `stock_ledger` for that `product_id`.
- Edit and Delete actions are admin-only (RBAC enforced at API level).

5. Add Product Dialog



Purpose

Create a new SKU with a configurable low-stock threshold.

Fields

FIELD	REQUIRED	DESCRIPTION
SKU	✓	Unique product code
Name	✓	Display name
Description	—	Optional details
Low Stock Threshold	✓	Per-product alert level (configurable per the user requirement)
Unit	✓	Unit of measure

Behavior

- Submitting saves the product and immediately recalculates dashboard stats.
- SKU uniqueness is enforced by a MongoDB unique index.

6. Location Management

Route: `/locations` (admin only)

Inventory Control

Admin (admin) Logout

Dashboard
Products
Locations
Stock Ledger

Location Management

Manage warehouse zones and storage bins

+ Add Location

WAREHOUSE	ZONE	aisle	BIN	CAPACITY	ACTIONS
WH1	A	1	01	100	
WH1	B	2	02	50	
WH2	C	3	03-updated	200	
WH1	A	1	01	100	
WH1	B	2	02	50	
WH2	C	3	03-updated	200	

Made with Emergent

Purpose

Manage warehouse zones, aisles, and storage bins.

Hierarchy Model

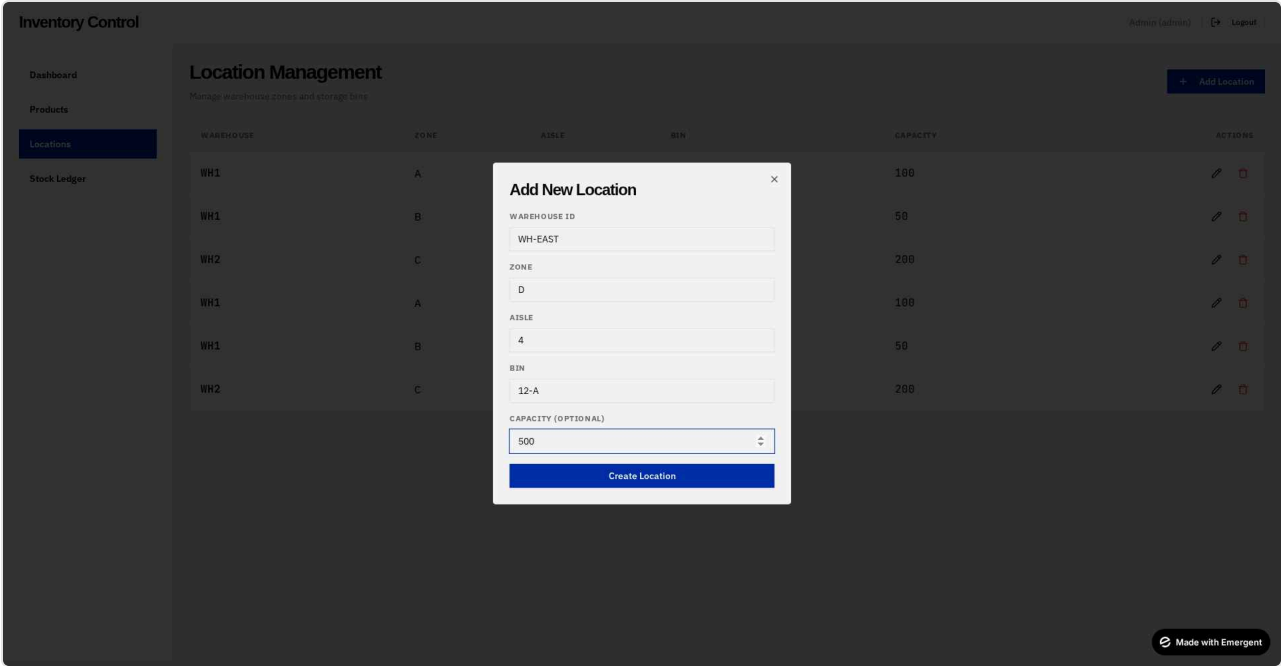
Locations follow a 4-part structure that supports future pick-path optimization:

```
Warehouse → Zone → Aisle → Bin
WH1      →  A  →   1   → 01
```

Table Columns

- **Warehouse** — facility identifier (mono)
- **Zone** — high-level area (A, B, C...)
- **Aisle** — numeric aisle within zone
- **Bin** — specific shelf/slot
- **Capacity** — optional metadata (max units the bin holds)
- **Actions** — Edit / Delete

7. Add Location Dialog



Purpose

Register a new storage location.

Fields

FIELD	REQUIRED	NOTES
Warehouse ID	✓	e.g., WH-EAST , WH1
Zone	✓	e.g., A , B , D
Aisle	✓	e.g., 1 , 4
Bin	✓	e.g., 01 , 12-A
Capacity	—	Optional max-units integer

8. Stock Ledger

Route: `/stock-ledger` (admin only)

Inventory Control

Admin (admin) Logout

DashboardProductsLocationsStock Ledger

Stock Ledger

Immutable transaction history

+ Record Transaction

Timestamp	Type	Product	Location	Quantity	Reference
5/24/2026, 11:37:11 AM	AUDIT	TEST_SKU_d93a5c - TEST Widget	WH1-A-1-01	+1	-
5/24/2026, 11:37:11 AM	PICK	TEST_SKU_d93a5c - TEST Widget	WH1-A-1-01	-5	-
5/24/2026, 11:37:10 AM	RECEIVE	TEST_SKU_d93a5c - TEST Widget	WH1-A-1-01	+100	PO-1

Made with Emergent

Purpose

Immutable transaction history — the single source of truth for all inventory movements.

Architecture Note

This is the **heart of the system**. Unlike traditional inventory systems that mutate a `quantity` field, every stock change is appended as a new ledger row:

- No UPDATE operations
- No DELETE operations
- Current stock is always derived: `current_stock = SUM(quantity_change WHERE product_id = X)`

This guarantees:

- Full audit trail
- Time-travel queries (stock at any past timestamp)
- Reconciliation simplicity
- Conflict-free mobile offline sync (transactions are append-only)

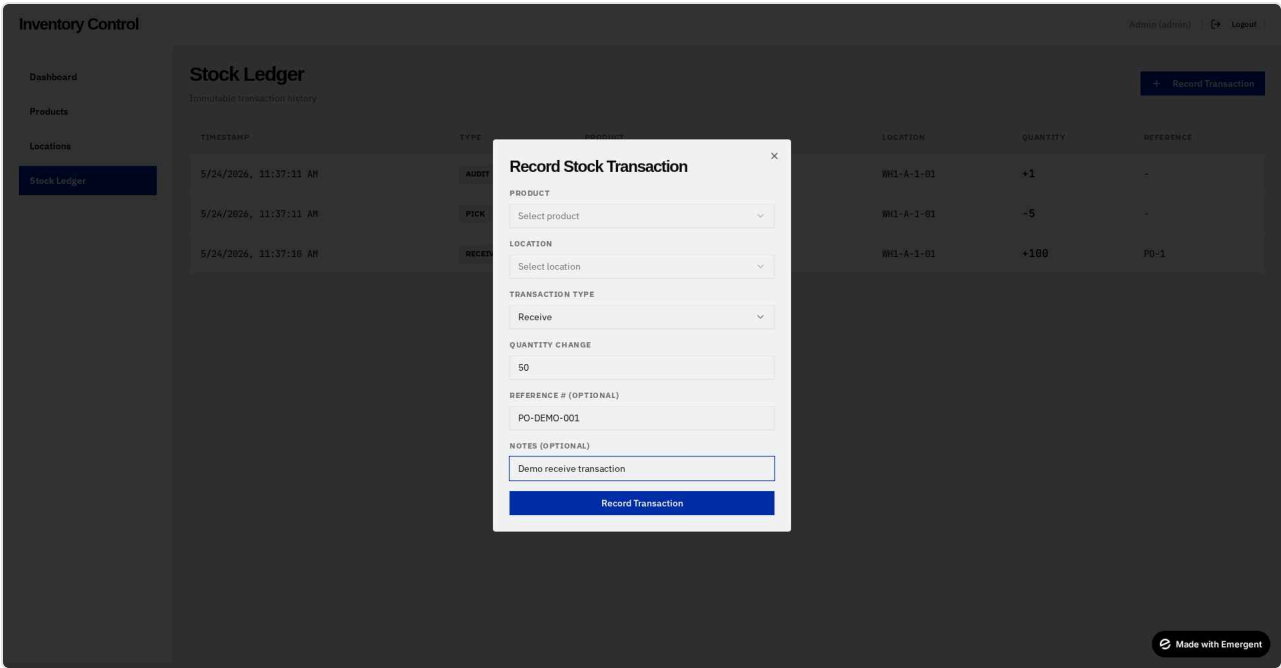
Table Columns

COLUMN	DESCRIPTION
Timestamp	Server-side <code>datetime.now(timezone.utc)</code>
Type	Badge: <code>RECEIVE</code> · <code>PICK</code> · <code>TRANSFER</code> · <code>AUDIT</code>
Product	SKU + product name
Location	Hierarchical: <code>WH1-A-1-01</code>
Quantity	Signed delta (<code>+100</code> , <code>-5</code> , <code>+1</code>)
Reference	Optional external ID (PO#, order#, etc.)

Transaction Types

TYPE	SIGN	USE CASE
<code>RECEIVE</code>	<code>+</code>	Stock arrives from supplier
<code>PICK</code>	<code>-</code>	Stock leaves for an order
<code>TRANSFER</code>	<code>±</code>	Move between locations (two paired entries)
<code>AUDIT</code>	<code>±</code>	Physical count reconciliation adjustment

9. Record Transaction Dialog



Purpose

Manually record a stock movement (also used by the mobile app via `/api/sync/push`).

Fields

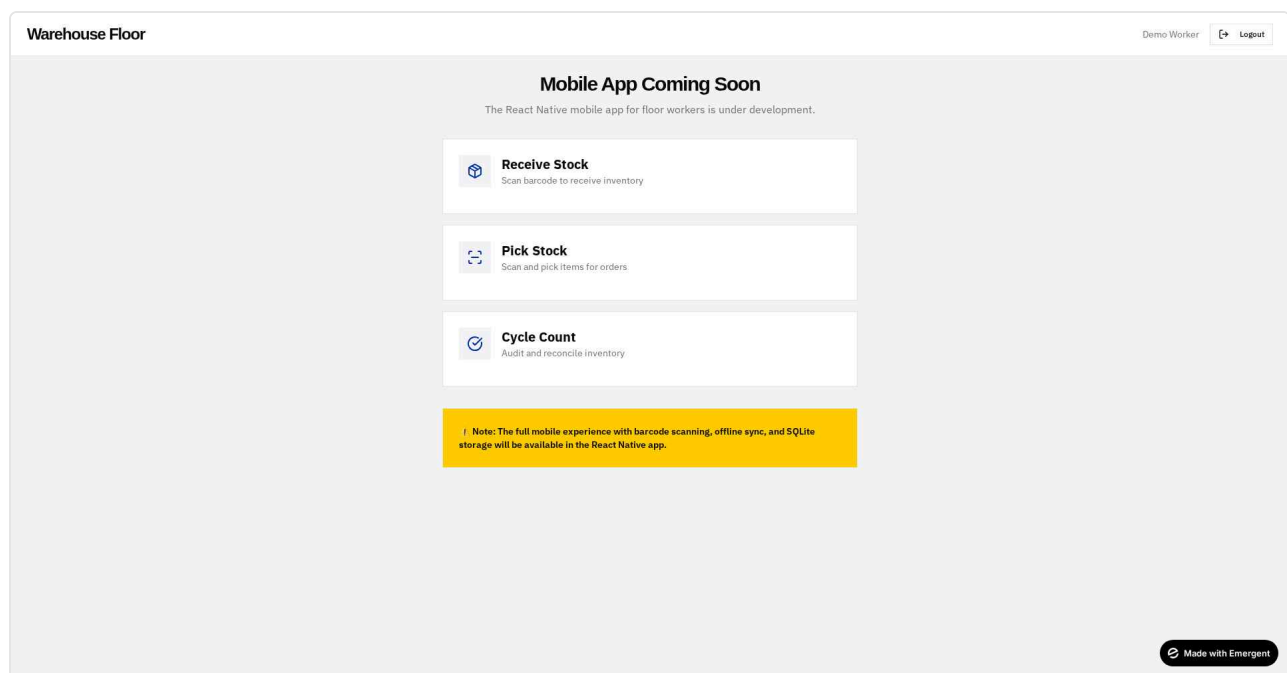
FIELD	REQUIRED	DESCRIPTION
Product	✓	Searchable select of all SKUs
Location	✓	Searchable select of all bins
Transaction Type	✓	Receive / Pick / Transfer / Audit
Quantity Change	✓	Signed integer (positive for receive, negative for pick)
Reference #	—	Optional — links to PO/order/audit ID
Notes	—	Free-form context

Behavior

- POST `/api/stock/transaction` → inserts a single new row in `stock_ledger`
- Dashboard stats auto-recompute on next refresh
- Linked to authenticated user (recorded in `user_id` field)

10. Worker Page

Route: `/worker` (worker role)



Purpose

Landing page for warehouse floor workers — interim placeholder until the React Native mobile app ships.

Key Elements

Three Feature Cards

Each card represents a planned mobile flow:

- **Receive Stock** — Scan barcode to receive inventory
- **Pick Stock** — Scan and pick items for orders
- **Cycle Count** — Audit and reconcile inventory

Warning Banner (yellow)

Communicates that the full mobile experience (barcode scanning, offline-first SQLite sync, large tap targets) is coming in the React Native build documented at `/app/mobile/`.

Header Difference

Note the header reads "**Warehouse Floor**" instead of "Inventory Control" — visual cue that this is a worker context, not an admin one.

Architecture Highlights

Visible in the UI

FEATURE	WHERE TO SPOT IT
Immutable StockLedger	Stock Ledger page — no edit/delete buttons; <code>Current Stock</code> on Products page is always derived from aggregation
Role-Based Access Control	Admin sees Dashboard/Products/Locations/Ledger sidebar; Worker is routed to the Warehouse Floor page only
Configurable thresholds	Add Product → "Low Stock Threshold" field per SKU
Real-time aggregation	Dashboard <code>Total Stock</code> is <code>SUM(stock_ledger.quantity_change)</code> computed on every request
Swiss design system	Cabinet Grotesk headings · IBM Plex Sans body · JetBrains Mono for SKUs/quantities · monochrome surfaces with single blue (<code>#002FA7</code>) + red (<code>#FF3B30</code>) accents · no shadows, sharp 1px borders

Backend API Endpoints

ENDPOINT	METHOD	AUTH	PURPOSE
/api/auth/register	POST	–	Create user
/api/auth/login	POST	–	Authenticate
/api/auth/logout	POST	✓	Clear session
/api/auth/me	GET	✓	Current user
/api/products	GET / POST	✓ / admin	List / create
/api/products/{id}	PUT / DELETE	admin	Update / delete
/api/locations	GET / POST	✓ / admin	List / create
/api/locations/{id}	PUT / DELETE	admin	Update / delete
/api/stock/transaction	POST	✓	Insert ledger row
/api/stock/ledger	GET	✓	List transactions
/api/stock/product/{id}	GET	✓	Per-product history
/api/dashboard/stats	GET	✓	KPI aggregations
/api/dashboard/low-stock	GET	✓	Low-stock alerts
/api/sync/push	POST	✓	Mobile bulk upload
/api/sync/pull	GET	✓	Mobile fetch updates

Data Model

```
users      { _id, email, password_hash, name, role, created_at }
products   { id, sku, name, description, low_stock_threshold, unit, created_at }
locations  { id, warehouse_id, zone, aisle, bin, capacity, created_at }
stock_ledger { id, product_id, location_id, user_id, transaction_type,
               quantity_change, reference_number, notes, timestamp }
```

Quick Demo Script (for 5-min video)

TIME	SECTION	WHAT TO SHOW
0:00 – 0:30	Intro	Title card + "Inventory management with immutable ledger architecture"
0:30 – 1:00	Login	Show Swiss design login page → sign in as admin
1:00 – 1:45	Dashboard	Walk through 4 KPI cards, hover over low-stock alerts, scroll to recent transactions
1:45 – 2:30	Products	Show table → click Add Product → demonstrate threshold field → save → see it appear
2:30 – 3:00	Locations	Show hierarchy (Warehouse → Zone → Aisle → Bin) → add a location
3:00 – 3:45	Stock Ledger	Highlight "immutable" badge — no edit/delete → Record Transaction → RECEIVE 100 units → show ledger row appear + Dashboard stats update
3:45 – 4:15	Worker view	Logout → register as worker → land on Warehouse Floor page → explain mobile roadmap
4:15 – 4:45	API tour	Quick Postman/curl showing same flows via API (admin authorization required for product/location CRUD)
4:45 – 5:00	Outro	Tech stack recap (FastAPI + React + MongoDB) + "next: React Native mobile with barcode scanning + SQLite offline sync"